Getting the model right is more important at this stage. As you'll see, selection of data types doesn't always fully capture the *essence* of all the properties of the model, even in this small application, so that's what we'll tackle next.

What's with turbo_method: :delete?

You may have noticed that the Destroy link includes the `turbo_method: :delete` parameter. This parameter determines which method is called in the `ProductsController` class and also affects which HTTP method is used.

Browsers use HTTP to talk with servers. HTTP defines a set of verbs that browsers can employ and defines when each can be used. A regular hyperlink, for example, uses an HTTP `GET` request. A `GET` request is defined by HTTP as a means of retrieving data and therefore isn't supposed to have any side effects. Using the `method` parameter in this way indicates that an HTTP `DELETE` method should be used for this hyperlink. Rails uses this information to determine which action in the controller to route this request to.

Note that when used within a browser, Rails substitutes the HTTP `POST` method for `PUT`, `PATCH`, and `DELETE` methods and in the process tacks on an additional parameter so that the router can determine the original intent. Either way, the request isn't cached or triggered by web crawlers.

Playtime

Here's some stuff to try on your own:

- We created tables in our database using a migration. Try examining the tables directly by running `bin/rails dbconsole`. This will put you directly into the SQLite database that the app uses. Type `.help` and hit `Return` to see the commands you can run to examine the database. If you know SQL, you can execute SQL in here as well.

- If you're feeling frisky, you can experiment with rolling back the migration. Type the following:

  ```
  depot> bin/rails db:rollback
  ```

  Your schema will be transported back in time, and the `products` table will be